PRISM Document

PowerPulse Electricity Forecasting, Spike Detection & Appliance Models

Document type	PRISM model document (Purpose, Resources, Implementation, Scoring, Maintenance)
Models covered	Model A: whole-home next-hour forecaster and spike detector (XGBoost). Model B: per-appliance hourly energy models for REFIT houses (Random Forest).
Source material	Two uploaded Colab notebooks: Electricity_Consumption_Forecasting_and_Spike_Detection.ipynb and PowerPulse_AllHouses_Colab.ipynb.
Not reviewed	The shared Colab link requires a Google sign-in and could not be opened. Neither notebook contains saved outputs, so no measured results are reported.

How to read this document. PRISM is used here as a five-part structure: **P**urpose and scope, **R**esources (data and environment), **I**mplementation (pipeline and model), **S**coring (evaluation), and **M**aintenance, monitoring and risks. Each part covers Model A and Model B.

1. Model Overview

	Model A: Forecast & Spike Detection	Model B: PowerPulse Appliance Models
Question answered	How much electricity will the home use in the coming hours, days and month, and which hours look unusually high?	How much energy does each individual appliance use in a given hour, given whole-house readings?
Algorithm	XGBRegressor (gradient-boosted trees)	RandomForestRegressor, one model per appliance per house
Granularity	Hourly, single whole-home series	Hourly, houses 1 to 21 (House14 has no mapping)
Target	Next-hour Hourly_energy_kWh	appliance{i}_kwh for each mapped appliance
Outputs	7-day and 30-day forecasts, spike flags, interactive chart, 4 CSV files	all_houses_results.csv and summary_table.csv with train/test MSE, MAE and R2

2. P: Purpose and Scope

2.1 Model A: Forecasting and spike detection

- Train a model that predicts the next hour of household electricity consumption from recent history and calendar information.
- Extend the model recursively to produce a 7-day (168 hours) and a 30-day (720 hours) forecast.
- Flag forecast hours that look like potential spikes compared with the historical behaviour of the same hour of day.
- Present results in an interactive weekly/monthly chart and export them as CSV files.

2.2 Model B: Appliance-level models

- Process every REFIT house file and train a separate model for each metered appliance.
- Compare training error with testing error (MSE ratio) to judge whether a model generalises or overfits.
- Produce a combined results table and summary statistics across all houses.

2.3 Out of scope

- Model A does not guarantee that flagged hours will be real spikes; the notebook itself states this.
- Neither notebook covers deployment, serving, automated retraining or alerting.

3. R: Resources

3.1 Model A: data and environment

Item	Detail
Input file	MyDrive/Electricity_consumption/hourly_power_consumption.csv
Required columns	Datetime and Hourly_energy_kWh
Cleaning	Datetimes and values coerced to proper types, rows with missing values dropped, data sorted by time, duplicate timestamps averaged.
Minimum history	At least 168 hourly readings are needed to forecast (one week of lags).
Libraries	pandas, numpy, matplotlib, xgboost, scikit-learn (metrics), ipywidgets
Platform	Google Colab with Google Drive mounted
Outputs saved to	Same Drive folder: weekly_electricity_forecast.csv, monthly_electricity_forecast.csv, weekly_predicted_spikes.csv, monthly_predicted_spikes.csv

3.2 Model B: data and environment

Item	Detail
Dataset	REFIT household electricity data: one CSV per house (House1.csv, House2.csv, and so on) in a REFIT folder.
Raw format	Unix timestamp, Aggregate, Appliance1 to Appliance9, Issues
Appliance mapping	A dictionary maps each house to its appliance names (for example House1: Fridge, Freezer(1), Freezer(2), Washer Dryer, and so on). Houses 1 to 13 and 15 to 21 are mapped.
Skip list	House12 appliances 7, 8 and 9 recorded nothing and are excluded from the impossible-hour check.
Cleaning function (defined)	Removes duplicate and invalid rows, masks negative readings, drops rows flagged in Issues, aggregates to minutes, then to hours. An hour needs at least 45 minutes of data. Adds max, min and std per hour and converts watts to kWh.
Input actually used by the run cell	Already-cleaned hourly files. The Step 8 loop skips cleaning and reads each file with the timestamp as index. File names with CLEAN_ or _hourly are normalised to HouseN.
Libraries	pandas, numpy, matplotlib, scikit-learn

Item	Detail
Paths	Input ./REFIT, output ./PowerPulse_AllHouses_Results (local version)

4. I: Implementation

4.1 Model A pipeline

The notebook runs in this order: install libraries, mount Drive, load and clean data, build features, split, train, evaluate, forecast, flag spikes, chart, export.

Feature	Definition
lag_1, lag_2, lag_3	Consumption 1, 2 and 3 hours back
lag_24, lag_168	Consumption 1 day and 1 week back
rolling_mean_24, rolling_mean_168	Average of the previous 24 and 168 hours (computed on the series shifted by one hour)
hour, day_of_week, month	Calendar features from the row timestamp
Target	target_next_hour, the value one hour ahead of the row

Setting	Value
Split	Chronological: earliest 80% train, latest 20% test (no shuffling)
Algorithm	XGBRegressor
Hyperparameters	n_estimators 300, max_depth 6, learning_rate 0.05, objective reg:squarederror, random_state 42, n_jobs -1

4.2 Model A: recursive forecast

- The function generate_forecast(horizon) predicts one hour, appends that prediction to the history, then predicts the next hour, repeating for 168 or 720 steps.
- Predictions are floored at zero.
- The weekly and monthly forecasts both start from the same last observed data and are generated independently.
- The notebook warns that errors accumulate over long horizons, so the 30-day forecast should be treated as an estimate.

4.3 Model A: spike rule

For each hour of day, the notebook computes the historical median and the median absolute deviation (MAD). A forecast hour is flagged as a Potential Spike when:

`prediction > median + 3 x 1.4826 x MAD`

The factor 1.4826 scales MAD to be comparable with a standard deviation, so the rule behaves like a robust 3-sigma threshold per hour of day.

4.4 Model B pipeline

Stage	Detail
Lag features	Aggregate kWh at lags 1, 2, 3; each appliance kWh at lags 1, 2, 3; rolling mean of aggregate over 3 and 6 hours

Stage	Detail
Time features	hour, weekday, month, is_weekend, hour_sin, hour_cos
Targets	appliance{i}_kwh for each appliance in the house mapping
Feature set	All columns except the targets and the house label
Row filtering	Drop hours missing aggregate or any target; drop impossible hours where the appliance sum exceeds 1.10 x aggregate + 0.005 kWh
Split	Chronological 80% train, 20% test, per house
Model	RandomForestRegressor: 200 trees, min_samples_leaf 3, random_state 42, n_jobs -1; predictions clipped at zero
Per-appliance metrics	Train and test MSE, MAE, R2; MSE ratio (test divided by train); mean kWh in train and test
Flag	MSE ratio below 2.0 is labelled comparable; otherwise possible overfitting

5. S: Scoring and Evaluation

5.1 What each notebook measures

	Model A	Model B
Metrics	MAE, MSE, RMSE, R2 on the held-out 20%	Train and test MSE, MAE, R2; MSE ratio
Visual check	Actual vs predicted plot for up to the first 168 test hours	None; results are tabular
Aggregate summary	Single set of scores	Mean and median MSE ratio and R2; counts of appliances with ratio below 2.0, test R2 above 0.9, and test R2 above 0.5
Spike evaluation	None. Spike flags are not scored against real events.	Not applicable

5.2 Results

Neither notebook was supplied with saved outputs, and the underlying data files were not provided, so no metric values are claimed here. After running the notebooks, record the results below.

Metric	Value	Run date / notes
Model A: MAE (kWh)		
Model A: RMSE (kWh)		
Model A: R2		
Model A: weekly potential spikes		
Model A: monthly potential spikes		
Model B: mean test R2		
Model B: median MSE ratio		

Metric	Value	Run date / notes
Model B: appliances with ratio below 2.0		

6. M: Maintenance, Monitoring and Risks

6.1 Points to verify before relying on the results

Model A: feature timing may differ between training and forecasting. In training, each row's features describe the hour before the row's own timestamp, while its target is the hour after. For example, lag_1 is 2 hours before the target and hour is the hour of the row, not of the target. In generate_forecast, lag_1 is the hour immediately before the predicted hour and hour is the predicted hour's own value. If this reading is right, the evaluation scores and the future forecasts are produced under slightly different feature definitions. Recommended check: align training features with the forecast function, for example by using the target hour's calendar features and shifting lags so lag_1 is the hour directly before the target, then re-evaluate.

Model B: possible target leakage. The feature set is every column except the target kWh columns and the house label. If the cleaned files include the appliance power columns (appliance{i}_w) and their max, min and std columns, these describe the same appliance in the same hour as the target and would reveal it. The target appliance's own lag columns are also included. Recommended check: print feature_cols for one house and exclude every column derived from the target appliance in the current hour; decide deliberately whether the appliance's own lags are allowed.

Model B: training scores are in-sample. Random forests fit training data closely, so train MSE is usually very low and the MSE ratio can look poor even for reasonable models. The 2.0 cut-off is a rule of thumb, not a statistical test.

6.2 Known limitations

- Recursive forecasting compounds error; the 30-day forecast is much less reliable than the 7-day forecast.
- Model A has no weather, holiday, occupancy or tariff inputs, and the month feature cannot learn seasonality from less than a year of data.
- Model A uses a single 80/20 split. A rolling or expanding backtest would give a more dependable error estimate.
- Spike flags are statistical outliers relative to history, not confirmed events, and are not validated against actual spikes.
- Model B trains separate models per appliance with no hyperparameter tuning and a single split per house.
- Model B appliance mappings contain placeholders (for example Unknown_2, Unknown_3, Unknown_5, Unknown_7) whose results are hard to interpret.
- Model B notebook headings mention Google Drive, but the code uses local paths; the cleaning function is defined but not called in the run loop.

6.3 Maintenance recommendations

Area	Recommendation
Retraining	Retrain Model A on a regular schedule (for example monthly) so recent behaviour is reflected. Retrain Model B when appliances change.
Monitoring	Track rolling MAE on new actuals for Model A and compare with the test MAE. Re-check spike thresholds when consumption patterns shift.

Area	Recommendation
Reproducibility	Fixed random seeds are set (42). Record library versions and data date ranges with every run.
Data quality	Validate the CSV schema, handle missing hours explicitly (Model A currently assumes a continuous hourly index), and log dropped rows.
Privacy	Consumption data can reveal household behaviour. Keep data and CSV outputs private; sharing the notebook does not share the data.
Documentation	Save notebook outputs (metrics, plots) with each version so results can be compared across runs.

7. Quick Run Guide

Model A

- Place hourly_power_consumption.csv in MyDrive/Electricity_consumption with columns Datetime and Hourly_energy_kWh.
- Open the notebook in Colab, mount Drive, and run all cells from top to bottom.
- Use the dropdown to switch between the weekly and monthly forecast; find CSV files in the same Drive folder.

Model B

- Put the house CSV files in a REFIT folder next to the notebook (cleaned hourly files, as the run cell expects).
- Run all cells; review all_houses_results.csv and summary_table.csv in PowerPulse_AllHouses_Results.